Labsheet 7 : BTCS301P-AS07

Computer Vision Lab

Khushi Choudhary CU240251564

In [ ]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import display, clear_output, Image

video_path = 'sample_video.mp4'
cap = cv2.VideoCapture(video_path)
if not cap.isOpened():
    print("Error: Video file could not be opened.")
else:
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    fps = cap.get(cv2.CAP_PROP_FPS)
    width  = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    print(f"Loaded Video: {total_frames} total frames | {fps} FPS | Resolution:
```

```
Loaded Video: 115 total frames | 25.0 FPS | Resolution: 3840x2160
```

In [ ]:
```
cap = cv2.VideoCapture(video_path)

# VideoWriter setup to output full sequence
fourcc = cv2.VideoWriter_fourcc(*'mp4v')
out_lk = cv2.VideoWriter('lucas_kanade_output.mp4', fourcc, fps, (width, height)
feature_params = dict(maxCorners=100, qualityLevel=0.3, minDistance=7, blockSize
lk_params = dict(winSize=(15, 15), maxLevel=2,
                 criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10,

color = np.random.randint(0, 255, (100, 3))
ret, old_frame = cap.read()
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)
p0 = cv2.goodFeaturesToTrack(old_gray, mask=None, **feature_params)
mask = np.zeros_like(old_frame)

frame_idx = 0
plt.figure(figsize=(12, 6))
while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    # Re-detect Shi-Tomasi corners if feature points vanish
    if p0 is None or len(p0) < 10:
        p0 = cv2.goodFeaturesToTrack(old_gray, mask=None, **feature_params)

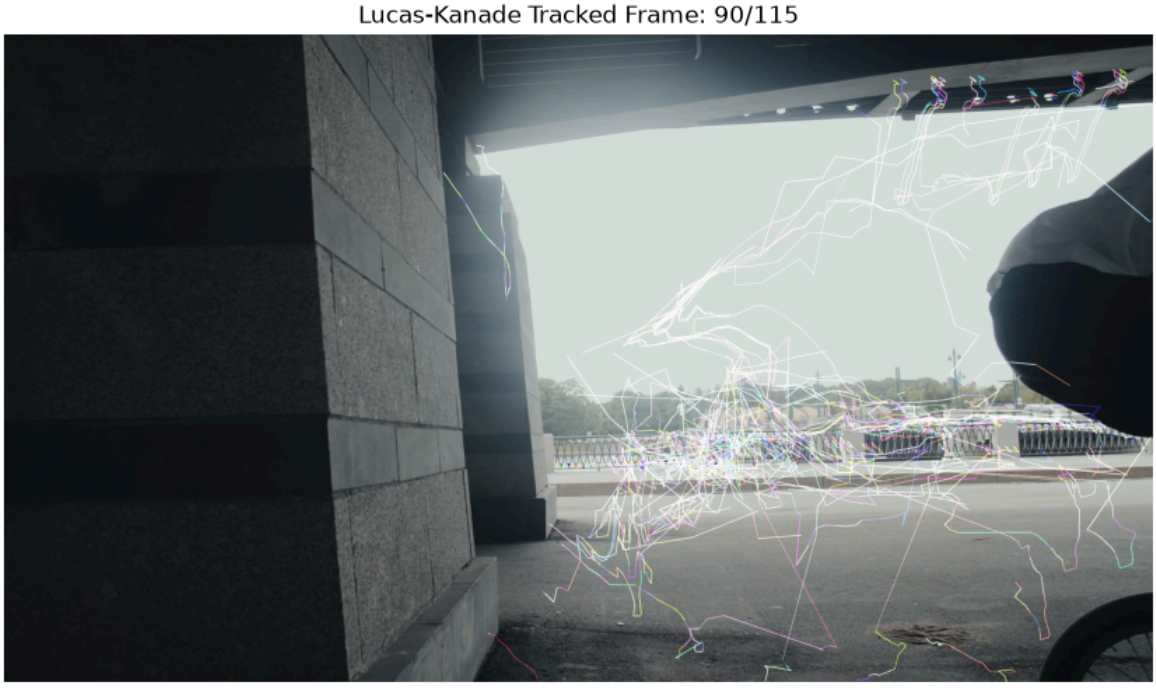
    if p0 is not None:
        p1, st, err = cv2.calcOpticalFlowPyrLK(old_gray, frame_gray, p0, None, *

        if p1 is not None:
            good_new = p1[st == 1]
            good_old = p0[st == 1]
```

```
            for i, (new, old) in enumerate(zip(good_new, good_old)):
                a, b = new.ravel()
                c, d = old.ravel()
                mask = cv2.line(mask, (int(a), int(b)), (int(c), int(d)), color[
                frame = cv2.circle(frame, (int(a), int(b)), 4, color[i % 100].to
            p0 = good_new.reshape(-1, 1, 2)

    img = cv2.add(frame, mask)
    out_lk.write(img)
    old_gray = frame_gray.copy()
    # Display snapshot every 30 frames
    if frame_idx % 30 == 0:
        clear_output(wait=True)
        plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
        plt.title(f"Lucas-Kanade Tracked Frame: {frame_idx}/{total_frames}")
        plt.axis('off')
        display(plt.gcf())
    frame_idx += 1

cap.release()
out_lk.release()
plt.close()
print(f"Sparse Flow Finished: Saved {frame_idx} frames to 'lucas_kanade_output.m
```




```
Sparse Flow Finished: Saved 114 frames to 'lucas_kanade_output.mp4'
```

In [6]:

```
cap = cv2.VideoCapture(video_path)
out_dense = cv2.VideoWriter('farneback_output.mp4', fourcc, fps, (width, height)

ret, frame1 = cap.read()
prvs = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
hsv = np.zeros_like(frame1)
hsv[..., 1] = 255

frame_idx = 0
plt.figure(figsize=(12, 6))

while cap.isOpened():
```

```
    ret, frame2 = cap.read()
    if not ret:
        break

    next_gray = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

    # Compute Farneback dense flow across full sequence
    flow = cv2.calcOpticalFlowFarneback(prvs, next_gray, None, 0.5, 3, 15, 3, 5,

    # Convert Cartesian motion vectors (u, v) to Polar (Magnitude, Angle)
    mag, ang = cv2.cartToPolar(flow[..., 0], flow[..., 1])
    hsv[..., 0] = ang * 180 / np.pi / 2
    hsv[..., 2] = cv2.normalize(mag, None, 0, 255, cv2.NORM_MINMAX)

    bgr_flow = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)
    out_dense.write(bgr_flow)

    prvs = next_gray

    # Display snapshot every 30 frames
    if frame_idx % 30 == 0:
        clear_output(wait=True)
        plt.imshow(cv2.cvtColor(bgr_flow, cv2.COLOR_BGR2RGB))
        plt.title(f"Farneback Dense Flow Frame: {frame_idx}/{total_frames}")
        plt.axis('off')
        display(plt.gcf())

    frame_idx += 1

cap.release()
out_dense.release()
plt.close()
print(f"Dense Flow Finished: Saved {frame_idx} frames to 'farneback_output.mp4'"
```




```
Dense Flow Finished: Saved 114 frames to 'farneback_output.mp4'
```